

# Contents

|                                               |          |
|-----------------------------------------------|----------|
| <b>BlockWarriors Game Development Guide</b>   | <b>2</b> |
| Table of Contents . . . . .                   | 2        |
| 1. Overview . . . . .                         | 2        |
| What is a Game? . . . . .                     | 2        |
| Key Concepts . . . . .                        | 2        |
| Game Lifecycle . . . . .                      | 2        |
| 2. Architecture . . . . .                     | 3        |
| File Structure . . . . .                      | 3        |
| Class Hierarchy . . . . .                     | 3        |
| 3. Getting Started . . . . .                  | 4        |
| Prerequisites . . . . .                       | 4        |
| Quick Start Checklist . . . . .               | 4        |
| 4. Implementing a New Game . . . . .          | 4        |
| Step 1: Create the Game Class . . . . .       | 4        |
| Step 2: Implement Abstract Methods . . . . .  | 5        |
| 5. Arena Configuration . . . . .              | 8        |
| YAML Format . . . . .                         | 8        |
| Accessing Arena Config in Your Game . . . . . | 9        |
| 6. Disconnect Handling . . . . .              | 10       |
| Disconnect Policies . . . . .                 | 10       |
| Implementation . . . . .                      | 10       |
| DisconnectResult Values . . . . .             | 11       |
| 7. Telemetry & Game State . . . . .           | 11       |
| What to Include . . . . .                     | 11       |
| Best Practices . . . . .                      | 11       |
| 8. Testing Your Game . . . . .                | 12       |
| Debug Commands . . . . .                      | 12       |
| Testing Workflow . . . . .                    | 12       |
| Local Testing Mode . . . . .                  | 13       |
| 9. Registration & Deployment . . . . .        | 13       |
| Step 1: Add Game Type Constant . . . . .      | 13       |
| Step 2: Update Shared Config . . . . .        | 13       |
| Step 3: Register in Plugin . . . . .          | 14       |
| 10. API Reference . . . . .                   | 14       |
| BaseGame Protected Fields . . . . .           | 14       |
| BaseGame Protected Methods . . . . .          | 14       |
| BaseGame Public Methods . . . . .             | 15       |
| 11. Examples . . . . .                        | 15       |
| Example: Point-Based Game . . . . .           | 15       |
| Example: Round-Based Game . . . . .           | 16       |
| Example: Cooperative Game . . . . .           | 16       |
| 12. Troubleshooting . . . . .                 | 17       |
| Common Issues . . . . .                       | 17       |
| Debug Logging . . . . .                       | 17       |
| Checklist Before Deployment . . . . .         | 17       |
| Appendix: Game Types Reference . . . . .      | 18       |

# BlockWarriors Game Development Guide

Version 1.0 | January 2026

A comprehensive manual for creating new game types for the BlockWarriors Beacon Minecraft plugin.

---

## Table of Contents

1. Overview
  2. Architecture
  3. Getting Started
  4. Implementing a New Game
  5. Arena Configuration
  6. Disconnect Handling
  7. Telemetry & Game State
  8. Testing Your Game
  9. Registration & Deployment
  10. API Reference
  11. Examples
  12. Troubleshooting
- 

## 1. Overview

### What is a Game?

In BlockWarriors, a “game” is a self-contained match type with its own rules, win conditions, and player interactions. Games are implemented as Java classes that extend the **BaseGame** abstract class.

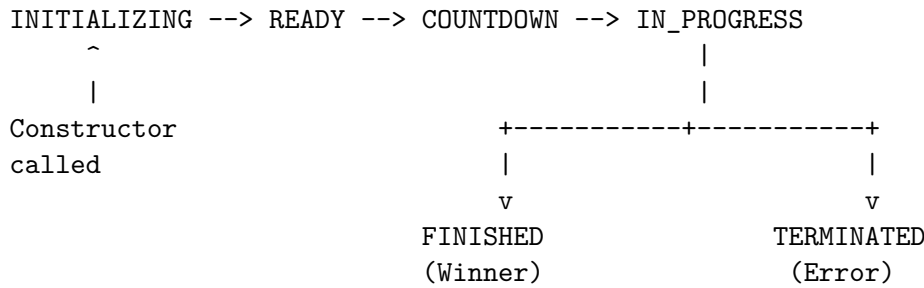
### Key Concepts

| Concept                 | Description                                                                              |
|-------------------------|------------------------------------------------------------------------------------------|
| <b>BaseGame</b>         | Abstract class defining the game contract                                                |
| <b>GameRegistry</b>     | Singleton that maps game types to factory functions                                      |
| <b>ArenaConfig</b>      | YAML-based arena configuration (spawns, boundaries, objectives)                          |
| <b>DisconnectPolicy</b> | How the game handles player disconnects                                                  |
| <b>GameState</b>        | Enum tracking game lifecycle (INITIALIZING → READY → COUNTDOWN → IN_PROGRESS → FINISHED) |

### Game Lifecycle

GAME LIFECYCLE

+=====+



## 2. Architecture

### File Structure

```

apps/blockwarriors-beacon/src/main/java/ai/blockwarriors/beacon/
+-- game/
|   +-- BaseGame.java           # Abstract base class
|   +-- GameFactory.java        # Factory interface
|   +-- GameRegistry.java       # Game type registry
|   +-- GameState.java          # State enum
|   +-- DisconnectPolicy.java   # Disconnect configuration
|   +-- DisconnectReason.java   # Why player disconnected
|   +-- DisconnectResult.java   # How game responds
|   +-- GracePeriodTracker.java # Grace period management
|   +-- impl/
|       +-- PvPGame.java        # Reference implementation
|       +-- SkyTilesGame.java   # Cooperative game example
+-- arena/
|   +-- ArenaManager.java       # Arena loading/management
|   +-- ArenaConfig.java        # Arena configuration POJO
|   +-- SpawnPoint.java         # Spawn point data
+-- constants/
|   +-- GameConfig.java         # Game type constants
+-- service/
|   +-- MatchManager.java       # Match lifecycle management
|   +-- MatchPollingService.java # Backend communication
|   +-- MatchTelemetryService.java # Real-time telemetry

apps/blockwarriors-beacon/src/main/resources/
+-- arenas/
|   +-- pvp.yml                 # PvP arena config
|   +-- sky_tiles.yml           # Sky Tiles arena config
+-- config.yml                  # Plugin configuration

```

### Class Hierarchy

BaseGame (abstract)

|                  |                                |
|------------------|--------------------------------|
|                  |                                |
| +++ PvPGame      | - 1v1 combat, instant forfeit  |
| +++ SkyTilesGame | - 2-player co-op, grace period |
| +++ WoolWarsGame | - 4v4 team game                |
| +++ BridgeGame   | - 1v1 racing                   |
| +++ [Your Game]  | - Custom implementation        |

---

### 3. Getting Started

#### Prerequisites

- Java 17+
- Maven
- Bukkit/Spigot API knowledge
- Understanding of the BlockWarriors backend (Convex)

#### Quick Start Checklist

- ☐ Create your game class extending `BaseGame`
  - ☐ Implement all abstract methods
  - ☐ Create an arena configuration file
  - ☐ Register your game in `Plugin.java`
  - ☐ Add game type to `GameConfig.java`
  - ☐ Update `game-config.json` in shared package
  - ☐ Test with `/testgame` command
- 

### 4. Implementing a New Game

#### Step 1: Create the Game Class

Create a new file in `beacon/game/impl/`:

```
package ai.blockwarriors.beacon.game.impl;

import ai.blockwarriors.beacon.arena.ArenaConfig;
import ai.blockwarriors.beacon.constants.GameConfig;
import ai.blockwarriors.beacon.game.*;

import org.bukkit.World;
import org.bukkit.entity.Player;
import org.bukkit.plugin.java.JavaPlugin;

import java.util.*;

/**
 * [Your Game Name] - Brief description
 */
```

```

* Game Rules:
* - Rule 1
* - Rule 2
*
* Win Condition:
* - How to win
*
* Disconnect Policy:
* - What happens when a player disconnects
*/
public class MyGame extends BaseGame {

    // Define your disconnect policy
    private static final DisconnectPolicy DISCONNECT_POLICY =
        DisconnectPolicy.instantForfeit(); // or withGracePeriod(30)

    // Game-specific state
    private boolean gameStarted = false;

    public MyGame(JavaPlugin plugin, String matchId) {
        super(plugin, GameConfig.GAME_TYPE_MY_GAME, matchId);
    }

    // Implement abstract methods (see below)
}

```

## Step 2: Implement Abstract Methods

**initialize()** Called once when the game is created. Set up initial state, register players, teleport to spawns.

```

@Override
public void initialize(World world, ArenaConfig arenaConfig,
    List<Player> blueTeamPlayers, List<Player> redTeamPlayers) {
    this.world = world;
    this.arenaConfig = arenaConfig;

    // Register players
    for (Player p : blueTeamPlayers) {
        players.add(p.getUniqueId());
        blueTeam.add(p.getUniqueId());
    }
    for (Player p : redTeamPlayers) {
        players.add(p.getUniqueId());
        redTeam.add(p.getUniqueId());
    }

    // Teleport to spawns
}

```

```

    teleportPlayersToSpawns(blueTeamPlayers, redTeamPlayers);

    // Set up players (inventory, effects, etc.)
    setupPlayers();

    setState(GameState.READY);
}

```

**start()** Called when the game should begin. Usually starts a countdown then transitions to IN\_PROGRESS.

```

@Override
public void start() {
    if (state != GameState.READY) {
        return;
    }

    setState(GameState.COUNTDOWN);
    startCountdown();
}

```

**handlePlayerDeath()** Handle a player death event. Return true if handled, false for default behavior.

```

@Override
public boolean handlePlayerDeath(Player deadPlayer, Player killer) {
    if (!isActive()) {
        return false;
    }

    // Game-specific death handling
    // Examples: respawn, eliminate, lose a life

    return true;
}

```

**handleObjective()** Handle game-specific objectives. Each game defines what objectives mean.

```

@Override
public boolean handleObjective(String objectiveType, Player player,
                               Map<String, Object> data) {
    switch (objectiveType) {
        case "flag_captured":
            return handleFlagCapture(player, data);
        case "point_scored":
            return handlePointScored(player, data);
        default:
            return false;
    }
}

```

```

    }
}

```

**checkWinCondition()** Check if the game should end. Called after significant events.

```

@Override
public boolean checkWinCondition() {
    // Example: First team to 5 points wins
    if (blueTeamScore >= 5) {
        winnerId = blueTeam.get(0);
        return true;
    }
    if (redTeamScore >= 5) {
        winnerId = redTeam.get(0);
        return true;
    }
    return false;
}

```

**getGameState()** Return current game state for telemetry. Include all relevant data.

```

@Override
public Map<String, Object> getGameState() {
    Map<String, Object> state = new HashMap<>();
    state.put("gameType", gameType);
    state.put("matchId", matchId);
    state.put("state", this.state.name());
    state.put("durationMs", getDurationMillis());

    // Game-specific data
    state.put("blueTeamScore", blueTeamScore);
    state.put("redTeamScore", redTeamScore);

    // Player stats
    List<Map<String, Object>> playerStats = collectPlayerStats();
    state.put("players", playerStats);

    if (winnerId != null) {
        state.put("winnerId", winnerId.toString());
    }

    return state;
}

```

**cleanup()** Clean up when game ends. Cancel tasks, clear effects, reset state.

```

@Override
public void cleanup() {

```

```

// Cancel any running tasks
if (gameTask != null && !gameTask.isCancelled()) {
    gameTask.cancel();
}

// Clear player effects
for (UUID playerId : players) {
    Player player = Bukkit.getPlayer(playerId);
    if (player != null && player.isOnline()) {
        for (PotionEffect effect : player.getActivePotionEffects()) {
            player.removePotionEffect(effect.getType());
        }
    }
}

LOGGER.info("Game cleanup completed for match " + matchId);
}

```

---

## 5. Arena Configuration

### YAML Format

Create a file in `src/main/resources/arenas/`:

```

# arenas/my_game.yml
name: "My Game Arena"
game_type: "my_game"
schematic: "my_game_arena.schem"

# Spawn points for each team
spawns:
  blue_team:
    - x: 10
      y: 65
      z: 0
      yaw: -90
      pitch: 0
  red_team:
    - x: -10
      y: 65
      z: 0
      yaw: 90
      pitch: 0

# Arena boundaries (for out-of-bounds detection)
boundaries:
  min:

```

```

    x: -50
    y: 50
    z: -50
max:
    x: 50
    y: 100
    z: 50

# Game-specific objectives/markers
objectives:
    blue_flag:
        x: 40
        y: 65
        z: 0
    red_flag:
        x: -40
        y: 65
        z: 0
    center:
        x: 0
        y: 65
        z: 0

# Optional: Game-specific settings
settings:
    max_score: 5
    respawn_delay_seconds: 3

```

## Accessing Arena Config in Your Game

```

// In initialize()
if (arenaConfig != null) {
    // Get spawn points
    List<SpawnPoint> blueSpawns = arenaConfig.getSpawns("blue_team");

    // Get boundaries
    Map<String, int[]> boundaries = arenaConfig.getBoundaries();
    int[] minBound = boundaries.get("min");
    int[] maxBound = boundaries.get("max");

    // Get objectives
    Map<String, int[]> objectives = arenaConfig.getObjectives();
    int[] flagPos = objectives.get("blue_flag");
}

```

## 6. Disconnect Handling

### Disconnect Policies

Choose a policy that fits your game:

| Policy                                   | Use Case                                 | Example   |
|------------------------------------------|------------------------------------------|-----------|
| <code>instantForfeit()</code>            | lv1 games where disconnecting is forfeit | PvP       |
| <code>withGracePeriod(seconds)</code>    | Allow reconnection attempts              | Sky Tiles |
| <code>teamGame(grace, minPlayers)</code> | Team games with player thresholds        | Wool Wars |

### Implementation

```
// Define policy
private static final DisconnectPolicy DISCONNECT_POLICY =
    DisconnectPolicy.withGracePeriod(30);

@Override
public DisconnectResult handlePlayerDisconnect(UUID playerId, DisconnectReason reason) {
    if (!hasPlayer(playerId)) {
        return null;
    }

    // Mark as disconnected
    disconnectedPlayers.put(playerId, System.currentTimeMillis());

    // Choose response based on your game's rules
    return DisconnectResult.GRACE_PERIOD; // or FORFEIT, CONTINUE, GAME_CANCELLED
}

@Override
public boolean handlePlayerReconnect(UUID playerId) {
    if (!isPlayerDisconnected(playerId)) {
        return false;
    }

    // Remove from disconnected
    disconnectedPlayers.remove(playerId);

    // Restore player state
    Player player = Bukkit.getPlayer(playerId);
    if (player != null) {
        respawnPlayer(player);
        return true;
    }
}
```

```

        return false;
    }

    @Override
    public DisconnectPolicy getDisconnectPolicy() {
        return DISCONNECT_POLICY;
    }

```

## DisconnectResult Values

| Result         | Behavior                                        |
|----------------|-------------------------------------------------|
| FORFEIT        | Game ends immediately, other team wins          |
| GRACE_PERIOD   | Wait for reconnection (uses GracePeriodTracker) |
| CONTINUE       | Game continues with remaining players           |
| GAME_CANCELLED | Game ends with no winner                        |

## 7. Telemetry & Game State

### What to Include

The `getGameState()` method should return data that:

- Tracks game progress
- Shows player statistics
- Enables spectator viewing
- Supports post-match analysis

### Best Practices

```

@Override
public Map<String, Object> getGameState() {
    Map<String, Object> state = new HashMap<>();

    // Required fields
    state.put("gameType", gameType);
    state.put("matchId", matchId);
    state.put("state", this.state.name());
    state.put("durationMs", getDurationMillis());

    // Game-specific stats
    state.put("round", currentRound);
    state.put("blueScore", blueTeamScore);
    state.put("redScore", redTeamScore);

    // Per-player stats
    List<Map<String, Object>> playerStats = new ArrayList<>();
    for (UUID playerId : players) {
        Map<String, Object> stats = new HashMap<>();
        stats.put("playerId", playerId.toString());
    }
}

```

```

        stats.put("team", getTeamForPlayer(playerId));
        stats.put("kills", getKills(playerId));
        stats.put("deaths", getDeaths(playerId));
        stats.put("disconnected", isPlayerDisconnected(playerId));

        Player player = Bukkit.getPlayer(playerId);
        if (player != null && player.isOnline()) {
            stats.put("health", player.getHealth());
            stats.put("position", locationToMap(player.getLocation()));
        }

        playerStats.add(stats);
    }
    state.put("players", playerStats);

    // Winner (if game ended)
    if (winnerId != null) {
        state.put("winnerId", winnerId.toString());
    }

    return state;
}

```

## 8. Testing Your Game

### Debug Commands

Use these commands to test your game locally:

| Command                      | Description                        |
|------------------------------|------------------------------------|
| /testgame <type>             | Start a test match without backend |
| /setarena <name>             | Load an arena for testing          |
| /endgame [winner]            | Force end the current game         |
| /triggerobjective <type>     | Simulate objective events          |
| /simulatedisconnect <player> | Test disconnect handling           |
| /simulatereconnect <player>  | Test reconnection                  |

### Testing Workflow

#### 1. Start a test game:

```
/testgame my_game
```

#### 2. Verify game state transitions:

- Check INITIALIZING → READY → COUNTDOWN → IN\_PROGRESS

#### 3. Test player events:

```
/triggerobjective flag_captured
```

#### 4. Test disconnect handling:

```
/simulatedisconnect PlayerName quit  
/simulatereconnect PlayerName
```

#### 5. End the game:

```
/endgame PlayerName
```

### Local Testing Mode

In config.yml:

```
development:  
  local-testing: true  
  skip-convex: false  
  auto-fill-bots: true
```

---

## 9. Registration & Deployment

### Step 1: Add Game Type Constant

In GameConfig.java:

```
public static final String GAME_TYPE_MY_GAME = "my_game";  
  
public static final String[] GAME_TYPES = {  
    GAME_TYPE_PVP,  
    GAME_TYPE_SKY_TILES,  
    GAME_TYPE_MY_GAME, // Add here  
    // ...  
};
```

### Step 2: Update Shared Config

In packages/shared/constants/game-config.json:

```
{  
  "gameTypes": {  
    "my_game": {  
      "id": "my_game",  
      "name": "My Game",  
      "description": "Brief description",  
      "tokensPerTeam": 1,  
      "players": "1v1"  
    }  
  }  
}
```

### Step 3: Register in Plugin

In Plugin.java:

```
@Override
public void onEnable() {
    // ... existing code ...

    GameRegistry registry = GameRegistry.getInstance();

    // Register your game
    registry.registerGame(
        GameConfig.GAME_TYPE_MY_GAME,
        MyGame::new,
        GameRegistry.GameMetadata.builder()
            .displayName("My Game")
            .description("Brief description")
            .playersPerTeam(1)
            .teamCount(2)
            .defaultArena("my_game")
            .build()
    );
}
```

---

## 10. API Reference

### BaseGame Protected Fields

| Field               | Type            | Description                     |
|---------------------|-----------------|---------------------------------|
| gameType            | String          | Game type identifier            |
| matchId             | String          | Unique match ID                 |
| plugin              | JavaPlugin      | Plugin reference for scheduling |
| world               | World           | Game world                      |
| arenaConfig         | ArenaConfig     | Arena configuration             |
| state               | GameState       | Current game state              |
| players             | Set             | All players                     |
| blueTeam            | List            | Blue team players               |
| redTeam             | List            | Red team players                |
| disconnectedPlayers | Map<UUID, Long> | Disconnected players            |
| startTime           | long            | Game start timestamp            |
| endTime             | long            | Game end timestamp              |
| winnerId            | UUID            | Winner player ID                |

### BaseGame Protected Methods

| Method                                  | Description                            |
|-----------------------------------------|----------------------------------------|
| <code>setState(GameState)</code>        | Update game state with logging         |
| <code>getOpponents(UUID)</code>         | Get players on opposing team           |
| <code>getTeammates(UUID)</code>         | Get teammates (excluding self)         |
| <code>countActivePlayers(String)</code> | Count non-disconnected players on team |
| <code>endGame(UUID)</code>              | End game with winner                   |
| <code>terminateGame(String)</code>      | Terminate game with reason             |

## BaseGame Public Methods

| Method                                  | Description                             |
|-----------------------------------------|-----------------------------------------|
| <code>getGameType()</code>              | Get game type string                    |
| <code>getMatchId()</code>               | Get match ID                            |
| <code>getState()</code>                 | Get current GameState                   |
| <code>getPlayers()</code>               | Get all players (unmodifiable)          |
| <code>getBlueTeam()</code>              | Get blue team (unmodifiable)            |
| <code>getRedTeam()</code>               | Get red team (unmodifiable)             |
| <code>hasPlayer(UUID)</code>            | Check if player is in game              |
| <code>getTeamForPlayer(UUID)</code>     | Get player's team ("blue" or "red")     |
| <code>isPlayerDisconnected(UUID)</code> | Check if player is disconnected         |
| <code>isActive()</code>                 | Check if game is IN_PROGRESS or PAUSED  |
| <code>hasEnded()</code>                 | Check if game is FINISHED or TERMINATED |
| <code>getDurationMillis()</code>        | Get game duration                       |
| <code>getWinnerId()</code>              | Get winner UUID                         |

## 11. Examples

### Example: Point-Based Game

```
public class PointGame extends BaseGame {
    private static final int WIN_SCORE = 10;
    private int blueScore = 0;
    private int redScore = 0;

    public void addScore(String team, int points) {
        if ("blue".equals(team)) {
            blueScore += points;
        } else {
            redScore += points;
        }
        broadcastScores();
    }

    @Override
```

```

    public boolean checkWinCondition() {
        if (blueScore >= WIN_SCORE) {
            winnerId = blueTeam.get(0);
            return true;
        }
        if (redScore >= WIN_SCORE) {
            winnerId = redTeam.get(0);
            return true;
        }
        return false;
    }
}

```

### Example: Round-Based Game

```

public class RoundGame extends BaseGame {
    private int currentRound = 0;
    private int maxRounds = 5;
    private int blueWins = 0;
    private int redWins = 0;

    public void endRound(String winningTeam) {
        if ("blue".equals(winningTeam)) blueWins++;
        else redWins++;

        currentRound++;

        if (currentRound >= maxRounds || blueWins > maxRounds/2 || redWins > maxRounds/2) {
            // Game over
            winnerId = blueWins > redWins ? blueTeam.get(0) : redTeam.get(0);
        } else {
            // Start next round
            startNextRound();
        }
    }
}

```

### Example: Cooperative Game

```

public class CoopGame extends BaseGame {
    private int teamLives = 5;
    private Set<UUID> playersAtGoal = new HashSet<>();

    @Override
    public boolean handlePlayerDeath(Player player, Player killer) {
        teamLives--;
        if (teamLives > 0) {

```

```

        respawnPlayer(player);
    }
    return true;
}

@Override
public boolean checkWinCondition() {
    // Win: All players reached goal
    if (playersAtGoal.containsAll(players)) {
        winnerId = blueTeam.get(0);
        return true;
    }
    // Lose: No lives remaining
    if (teamLives <= 0) {
        winnerId = null; // No winner
        return true;
    }
    return false;
}
}

```

## 12. Troubleshooting

### Common Issues

| Issue                        | Solution                                                                           |
|------------------------------|------------------------------------------------------------------------------------|
| Game not starting            | Check <code>state</code> is <code>READY</code> before calling <code>start()</code> |
| Players not teleporting      | Verify <code>ArenaConfig</code> has valid spawn points                             |
| Win condition not triggering | Ensure <code>checkWinCondition()</code> is called after relevant events            |
| Disconnect not handled       | Verify <code>handlePlayerDisconnect()</code> returns non-null                      |
| Telemetry not updating       | Check <code>getGameState()</code> returns valid data                               |

### Debug Logging

Add logging to track game flow:

```

LOGGER.info("Game " + matchId + " state: " + state);
LOGGER.info("Players: " + players.size() + ", Blue: " + blueTeam.size());
LOGGER.fine("Event handled: " + eventType);

```

### Checklist Before Deployment

- ☐ All abstract methods implemented
- ☐ Arena config created and tested
- ☐ Game type registered in `GameConfig.java`

- ☐ Game registered in `Plugin.java`
  - ☐ Shared config updated (`game-config.json`)
  - ☐ Tested with `/testgame` command
  - ☐ Tested disconnect/reconnect scenarios
  - ☐ Telemetry data verified
  - ☐ Win conditions tested
  - ☐ Edge cases handled (empty teams, simultaneous events)
- 

## Appendix: Game Types Reference

| Game      | Type ID                | Players        | Disconnect Policy |
|-----------|------------------------|----------------|-------------------|
| PvP       | <code>pvp</code>       | 1v1            | Instant forfeit   |
| Sky Tiles | <code>sky_tiles</code> | 2-player co-op | 30s grace period  |
| Wool Wars | <code>wool_wars</code> | 4v4            | 60s grace, min 2  |
| Bridge    | <code>bridge</code>    | 1v1            | Instant forfeit   |
| Bed Wars  | <code>bedwars</code>   | 4v4            | 60s grace, min 2  |
| CTF       | <code>ctf</code>       | 5v5            | 60s grace, min 3  |

---

*For questions or issues, contact the BlockWarriors development team.*